



UNIVERSITÀ DEGLI STUDI DI PALERMO

DIPARTIMENTO DI INGEGNERIA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

NexuStream

RELAZIONE DEL PROGETTO



Team di Sviluppo

Davide Alaimo
Claudio Luparello
Christian Manto

Supervisore

Luca Cruciata

Anno Accademico

2025 - 2026

Indice Generale

1	Introduzione	3
1.1	Proposta di progetto	3
2	Attori del sistema	3
2.1	Utente	3
2.2	Guest	3
2.3	Moderatore	3
2.4	Gestore catalogo	4
2.5	Admin	4
3	Architettura del sistema	4
4	Note sull'utilizzo dell'applicazione	4
5	Architettura del sistema (dettaglio)	6
5.1	Il Client (Frontend)	6
5.1.1	Src	6
5.2	Server (Backend)	21
5.2.1	Server.js	21
5.2.2	Routes	22
5.2.3	Middleware	22
5.2.4	Utils	23
5.2.5	Controllers	23
5.2.6	DB	24
5.2.7	Models	24
5.2.8	Public	24

1 Introduzione

1.1 Proposta di progetto

La nostra proposta è quella di realizzare un sito di **streaming anime** con un focus particolare sull'aspetto di **discussione**. Il nome del sito è ispirato proprio da questa filosofia: **Nexus** (in latino: “*nesso, legame, connessione*”), un **luogo centrale** dove riunirsi.

Il sito è pensato per **stimolare la discussione** sugli episodi degli anime appena rilasciati: i **commenti** verranno aperti non appena l'episodio sarà disponibile per la visione, mentre dopo **2 settimane** dall'uscita dell'episodio non sarà più possibile per gli utenti partecipare alla discussione. Questa scelta si adatta allo **standard dell'industria** che prevede solitamente il rilascio di episodi a **cadenza settimanale** fino al termine di una stagione. Inoltre è volta a permettere anche un'adeguata **moderazione** delle discussioni per il periodo in cui sono attive.

Nonostante ciò, prevediamo che una buona parte degli utenti potrebbe voler guardare le stagioni nella loro interezza al termine delle stesse. Per non precludere loro la possibilità di partecipare alla discussione, i commenti saranno **riaperti al termine di una stagione**, per un determinato periodo di tempo.

Il nostro obiettivo è quindi fare in modo di invogliare gli utenti a manifestare le proprie opinioni reali e **“a caldo”** su ciascuna serie, lasciando comunque una certa **libertà sui tempi di visione**.

Per casi particolari, si può prevedere in futuro il **blocco o sblocco manuale** delle discussioni da parte di utenti autorizzati.

Ogni episodio avrà una **sezione commenti** ad esso dedicata: i commenti supporteranno i **timestamp** del video, e **diversi stili di formattazione**. I commenti verranno gestiti come **thread**: sarà possibile quindi commentare l'episodio stesso o **rispondere** ad un commento particolare. L'intera thread farà sempre riferimento all'episodio.

2 Attori del sistema

Nei punti successivi di questo paragrafo sono elencati gli attori del sistema, caratterizzati da specifici permessi, obiettivi e modalità di interazione con l'applicazione.

2.1 Utente

L'Utente crea e gestisce il proprio profilo previa **registrazione** nella piattaforma. Può personalizzare l'avatar e impostare le preferenze di lingua per l'applicazione, l'audio e i sottotitoli; queste ultime influenzeranno la lingua predefinita dei contenuti visualizzati, se disponibile. Ha la facoltà di recuperare o modificare la password del proprio account, cercare show specifici per titolo o genere, aggiungere le serie ai **preferiti** tramite appositi pulsanti e interagire tramite “like” con episodi e commenti.

Può partecipare attivamente alle discussioni nelle rispettive sezioni commenti e interrompere la visione in qualsiasi momento, per poi riprenderla facilmente tramite la funzionalità **“Continua a guardare”**. Possiede i **privilegi minimi** di interazione all'interno dell'applicazione, superiori solo a quelli del Guest.

2.2 Guest

Il Guest rappresenta l'utente non autenticato. Può navigare ed esclusivamente visualizzare le sezioni e le pagine che non richiedono alcuna autorizzazione, come la **home**, la pagina di **ricerca** e la lista degli episodi di uno show. Non possiede l'autorizzazione per fruire dei contenuti in **streaming**, personalizzare profili, né per visualizzare o pubblicare commenti.

2.3 Moderatore

Il Moderatore presiede alla gestione delle interazioni tra gli utenti nelle sezioni commenti. In caso di violazione dei **ToS** (Termini di Servizio) della piattaforma, ha la facoltà (e il dovere) di **nascondere**

commenti offensivi e revocare il diritto di scrittura agli utenti, impedendo loro di commentare. Svolgerà le sue funzioni in parte all'interno della sezione commenti stessa, con un'**interfaccia modificata** appositamente e dotata di strumenti per facilitare le operazioni di moderazione (es. contrassegnare i commenti già approvati), e in parte da una pagina apposita riservata.

Può visualizzare la cronologia completa dei messaggi, inclusi quelli rimossi o nascosti. Ha inoltre la facoltà di gestire le discussioni, creandole o disattivandole all'occorrenza; sebbene vengano gestite automaticamente, il suo **intervento manuale** è fondamentale al fine di accomodare situazioni particolari, come aprire una discussione in corrispondenza di un evento speciale della community. Qualora lasci un commento, questo sarà contraddistinto da un **flair speciale** accanto allo username.

2.4 Gestore catalogo

Il Gestore Catalogo mantiene aggiornato il catalogo delle serie ed è responsabile dell'inserimento e del mantenimento dei contenuti multimediali. Opera tramite **schermate riservate** e particolari form dedicati per inserire, modificare o rimuovere serie, stagioni ed episodi, incluse le relative descrizioni. Ha il compito di aggiungere nuove lingue, tracce audio e sottotitoli agli episodi, oltre a gestire e aggiornare il catalogo degli **avatar** e delle foto profilo disponibili per gli utenti. Al pari del Moderatore, i suoi interventi pubblici all'interno della piattaforma sono identificati da un **flair speciale**.

2.5 Admin

L'Admin detiene i **privilegi massimi** e più alti di tutta l'applicazione. Si occupa della supervisione, della gestione e della nomina di Moderatori e Gestori Catalogo, che vengono selezionati da lui personalmente. Ha il pieno controllo sugli account, potendo visualizzare e **modificare lo status** di qualsiasi utente a suo piacimento. **Eredita e condivide interamente** tutte le funzionalità operative e le caratteristiche del Moderatore e del Gestore Catalogo. I suoi interventi sulla piattaforma sono distinti dal **flair più esclusivo** e speciale di tutti.

3 Architettura del sistema

Il progetto **NexuStream** adotta un'architettura di tipo **Client-Server**, configurata per garantire una netta separazione delle responsabilità, un'elevata scalabilità e una gestione centralizzata della sicurezza e persistenza dei dati. L'infrastruttura si articola in due macro-componenti indipendenti e disaccoppiate:

Frontend (Client): Sviluppato mediante il framework **Ionic** basato su **Angular**, deputato alla gestione dell'interfaccia utente, dell'esperienza di streaming e delle sezioni di discussione.

Backend (Server): Realizzato in ambiente Node.js utilizzando il framework **Express** in **JavaScript**, incaricato dell'esposizione delle API, della business logic e del controllo degli accessi.

La persistenza delle informazioni strutturate - quali dati degli utenti, relazioni dei thread e metadati del catalogo - è affidata a un database relazionale **SQLite**, integrato direttamente nel backend per garantire una gestione dei dati leggera ed efficiente.

La comunicazione tra client e server avviene esclusivamente tramite protocollo **HTTP/HTTPS**, dove lo scambio di informazioni standard è veicolato in formato **JSON**. Per le operazioni di caricamento dei media e degli asset grafici (come video, sottotitoli e avatar), il canale viene gestito tramite richieste **multipart/form-data**, elaborate lato server mediante il middleware **Multer**. I file così ricevuti vengono memorizzati in modo permanente all'interno del **file storage locale** del server, i cui percorsi (path) vengono indicizzati all'interno del database per consentire un recupero rapido dei contenuti durante lo streaming.

4 Note sull'utilizzo dell'applicazione

L'utilizzo dell'applicazione è in generale intuitivo, pensato per un utente qualsiasi. Le pagine riservate ai cataloguer, mod e admin sono pensate per essere utilizzate da utenti che sanno svolgere il proprio lavoro,

per cui si prevede che l'utilizzo dei form e delle feature ad essi disponibili sia più cauto e calcolato. In ogni caso, sono effettuati controlli per evitare che eventuali loro errori non creino danni irreversibili.

Un cataloguer non dovrebbe per esempio cancellare uno show senza prima avere cancellato tutte le sue stagioni, e a sua volta una stagione senza prima aver cancellato tutti i suoi episodi. Dovrebbe inoltre avere cura di caricare esclusivamente files del formato giusto, per esempio non caricare un'immagine verticale in un form che richiede un'immagine orizzontale.

Le discussioni vengono aperte automaticamente in due circostanze:

Un catalogatore carica un nuovo episodio. Si aprirà quindi automaticamente una discussione di 2 settimane.

Un catalogatore segna una stagione come completata. Si aprirà quindi una discussione di 1 mese per tutti gli episodi della stagione.

5 Architettura del sistema (dettaglio)

5.1 Il Client (Frontend)

Il client rappresenta l'interfaccia utente (UI) ed è stato sviluppato utilizzando il framework Ionic accoppiato ad Angular. Il suo compito è quello di gestire l'esperienza utente (UX) rendendo la navigazione fluida e reattiva, intercettare le azioni dell'utente, gestire lo stato dell'applicazione, memorizzare temporaneamente i dati di sessione nel LocalStorage del browser per evitare chiamate ridondanti al server.

Nei paragrafi successivi saranno descritti nel dettaglio le parti che compongono il frontend.

5.1.1 Src

La cartella **src** comprende tutti i file di sviluppo dell'applicazione che verranno successivamente compilati e ottimizzati per i dispositivi mobili o il browser. Di seguito viene analizzata l'organizzazione interna delle sue sottocartelle principali.

5.1.1.1 App

La cartella **app** costituisce il nucleo applicativo del frontend. Al suo interno, il codice è strutturato seguendo il paradigma della programmazione orientata ai componenti. La cartella è organizzata nelle seguenti macro-sezioni.

5.1.1.1.1 Components

Questa cartella contiene i componenti di NexuStream utilizzati in alcune pagine:

Avatar-picker-modal: questo componente descrive una finestra pop-up dove viene mostrata all'utente una lista di avatar da scegliere come immagine profilo insieme all'avatar corrente. Quando l'utente clicca su uno degli avatar, questo viene passato al metodo `select()` dello script che sostituisce l'avatar precedente. Infine l'utente, clicca sul bottone "Salva" che invoca il metodo `confirmSelection()` che chiude il modal passando l'avatar selezionato.

Cataloguer-modal: questo componente è formato da tre sottocomponenti: *cataloguer-show-modal*, *cataloguer-season-modal*, *cataloguer-episode-modal*. Il codice html di questi componenti descrivono tre form gestiti tramite una Modal (una finestra popup a schermo intero) a cui può accedere solo il catalogatore. In base alla modalità, gestita dalla variabile `isEditMode`, in cui si trova il catalogatore, quest'ultimo può modificare o aggiungere show, stagioni o episodi. La variabile `level` (che può valere `'shows'`, `'seasons'` o `'episodes'`), ricevuta in input dalla pagina principale, cambia istantaneamente l'interfaccia, i campi visibili e i testi, permettendo di gestire un unico form polimorfo senza dover creare tre form separati e duplicare il codice. Il form è legato alla logica di Angular tramite la direttiva `[formGroup]="cataloguerForm"`. I campi usano l'attributo `formControlName`, il che significa che la validazione dei campi obbligatori viene calcolata programmaticamente nel file TypeScript. Il pulsante di salvataggio finale, infatti, si disabilita automaticamente se i campi obbligatori non sono validi: `[disabled]="cataloguerForm.invalid"`. Il layout organizza visivamente i controlli separando i metadati testuali obbligatori in lingua italiana (titolo e trama principali) da quelli facoltativi dedicati alla localizzazione in lingua inglese, garantendo la coerenza dei dati immessi nel database. Il tag `<svg>` è stato usato dentro il pulsante di salvataggio per mostrare l'icona del floppy disk.

La logica di controllo dei tre componenti è affidata alle classi `CataloguerShowModalComponent`, `CataloguerSeasonModalComponent`, `CataloguerEpisodeModalComponent`, che si occupano di costruire la struttura dati del form, applicare le regole di validazione e gestire il parsing dei dati internazionalizzati. I componenti sfruttano le API reattive di Angular (`FormGroup` e `FormBuilder`) combinate con il pattern moderno di Dependency Injection tramite la funzione `inject()`. Per supportare l'internazionalizzazione nativa della piattaforma, i campi del database relativi a titoli e descrizioni sono strutturati in formato JSON. Il componente implementa un meccanismo di *fault-tolerant parsing* (`getLangText`) che estrae in modo sicuro le stringhe localizzate per popolare i controlli del form in base alla lingua dell'interfaccia client. Questa funzione intercetta la stringa, tenta di effettuarne il parsing in un oggetto JavaScript ed estrae solo la lingua che

serve in quel momento nel form. Se l'operazione fallisce (ad esempio se il testo è una stringa semplice), fa un fallback sicuro restituendo il testo grezzo senza rompere l'applicazione. In fase di inizializzazione (`ngOnInit`), i componenti valutano la presenza dell'input `"data"` per determinare il contesto operativo (creazione o editing). In modalità modifica, i validatori vengono rimodulati e i campi chiave (come `seasonNumber` o `episodeNumber`) vengono programmaticamente disabilitati per preservare l'integrità referenziale dei dati. In fase di sottomissione del modulo `"save()"` o l'estrazione del payload non avviene tramite la proprietà standard `".value"`, bensì tramite il metodo `".getRawValue()"`. Questa scelta garantisce l'inclusione nel modello dati anche dei controlli disabilitati, assicurando che i dati identificativi e temporali bloccati vengano correttamente trasmessi al backend Node.js per finalizzare l'operazione di persistenza. La validazione dei dati è gestita programmaticamente tramite i validatori sincroni di Angular. In particolare, l'utilizzo di `Validators.required` garantisce l'integrità referenziale dei dati prima del loro invio al backend, vincolando la sottomissione del form alla presenza obbligatoria di informazioni strutturali e testuali imprescindibili (come i titoli in lingua italiana, le date di rilascio o gli indici numerici di stagioni ed episodi). Nel file TypeScript, per sicurezza, è stato aggiunto un controllo blindato prima di chiudere la modale: `save() { if (this.cataloguerForm.invalid) return; }`

Change-password-modal: si tratta di una finestra modale asincrona che permette all'utente di modificare in sicurezza la propria password di accesso alla piattaforma NexuStream. Il form sfrutta l'iniezione delle API basate su controlli non nullabili (`NonNullableFormBuilder`) forzando il modulo a escludere automaticamente il valore null dai tipi dei controlli del modulo se il form viene resettato. Di conseguenza, quando viene invocato il metodo `this.passwordForm.getRawValue()`, si ottiene un oggetto le cui proprietà sono stringhe pure (`string`), azzerando errori di compilazione TypeScript o la necessità di effettuare controlli manuali sul tipo. Per garantire la congruenza dei dati inseriti, il ciclo di vita del form è vincolato al validatore personalizzato `passwordMatchValidator`. Operando a livello aggregato sul `FormGroup`, la routine esegue un controllo comparativo tra i campi di nuova password e conferma. Qualora venga rilevata una discrepanza, il sistema inietta programmaticamente un errore atomico (`mismatch`) all'interno del sotto-controllo di destinazione mediante il metodo `setErrors()`. L'attivazione del metodo `onSubmit` attiva un meccanismo di protezione dell'interfaccia utente tramite l'istanza di un componente `ion-loading`, escludendo l'eventualità di sottomissioni multiple concorrenti (*Race Conditions*). Il disaccoppiamento logico della chiamata API verso il micro-servizio `Settings` permette la gestione simmetrica degli esiti di rete: la risoluzione positiva produce la chiusura controllata della finestra con rilascio delle risorse, mentre l'intercettazione dei codici di errore HTTP esegue il parsing dinamico dei messaggi di rigetto del server, notificando l'anomalia all'utente per mezzo di toast informativi contestuali.

Comments: il template HTML di questo componente definisce la Sezione Commenti di NexuStream con Struttura a Thread. Esso è costituito da un sistema di interazione dinamico e gerarchico (commento principale e relative risposte annidate) dotato di anteprima in tempo reale, supporto al markup e un intero pannello di moderazione integrato contestualmente per amministratori e moderatori. Quando un utente clicca su "Rispondi", si attiva il blocco `@if (replyingToUsername)` in cima al form. L'interfaccia si adatta mostrando il banner "In risposta a @User", cambia il placeholder della textarea e inserisce automaticamente una `@mention` nell'anteprima. Se il commento è una risposta a sua volta, un clic sul tag della menzione invoca la funzione `scrollToComment()` per scorrere la pagina fino al messaggio originale. L'applicazione supporta anche la formattazione del testo nei commenti. Infatti, nel box di input è stata implementata un'anteprima live guidata dal Two-Way Data Binding (`[(ngModel)]`): `<span [innerHTML]="parseMarkdown(nuovoCommentoTesto)"></span>`. Il testo digitato viene elaborato a runtime dalla funzione `parseMarkdown()`. Per evitare di duplicare l'enorme blocco di codice HTML che definisce la struttura grafica del singolo commento (avatar, username, testo, bottoni "Like", "Segnala", ecc.) tra commenti principali e risposte viene utilizzato: `<ng-template #singleComment let-comment="comment" let-isReply="isReply" let-rootId="rootId">`. Il commento viene stampato una volta sola e, all'interno del ciclo `@for`, viene invocato dinamicamente passando contesti differenti tramite la direttiva astratta `*ngTemplateOutlet`: per il commento radice, `isReply: false`; per la risposta nel thread, `isReply: true`. Questo riduce le righe di codice, azzerando i bug da disallineamento grafico e alleggerisce il rendering del browser. La proprietà `track root.CommentID` all'interno del `@for` aggiornerà solo il commento che è cambiato.

Riguardo il pannello di moderazione, il template implementa una logica condizionale stringente per la

sicurezza e la pulizia della community, mostrando elementi diversi in base ai ruoli dell'utente loggato (`isMod`, `isAdmin`, `isCataloguer`):

Ruoli Visivi: Se un utente autorevole commenta, compaiono badge dedicati con icone Ionicons personalizzate.

Azioni di Moderazione Protette: Solo se `isMod || isAdmin` è vero, Angular esegue l'injection nel DOM del blocco `.mod-actions-group`. Questo blocco mostra i pulsanti per approvare al volo il commento (`toggleApprove`), nascondere dalla piattaforma (`toggleHide`) o bannare direttamente l'utente (`banUser`) richiamando la policy `canBanUser`.

Filtro di Visibilità: I commenti contrassegnati dal database come nascosti (`comment.isHidden`) non vengono scaricati nel DOM degli utenti comuni. Vengono renderizzati esclusivamente se chi guarda la pagina è un moderatore o un admin.

La classe `CommentsComponent` governa la logica di business legata alle interazioni della community e alla moderazione dei contenuti multimediali. L'architettura del componente si distingue per l'adozione di pattern avanzati di scomposizione delle strutture dati e la gestione della sicurezza a livello client. Nel ciclo di vita `ngOnInit`, il componente estrae autonomamente il JSON Web Token (`token`) dalla memoria locale del browser (`LocalStorage`). Senza effettuare chiamate sincrone aggiuntive al server, esegue una decodifica client-side del payload (`jwtDecodeHelper`) per estrarre l'ID utente (`currentUserId`) e verificare i privilegi amministrativi (`isAdmin`). Questo garantisce una reattività immediata nell'attivazione o disattivazione dei moduli grafici di moderazione. Questa architettura inoltre alimenta la policy `canBanUser`, la quale applica vincoli logici stringenti per impedire l'auto-sanzione e proteggere l'integrità dei membri dello staff da azioni di ban incrociate tra moderatori.

Poiché il database restituisce i commenti come una lista “piatta” (*flat list*), dove le risposte indicano semplicemente l'ID del messaggio a cui rispondono (`REF_CommentID`), all'interno di `caricaCommenti()`, è stato implementato un algoritmo efficiente per ristrutturare questa lista in una gerarchia “stile Instagram”, ovvero una struttura ad albero appiattita (tutte le risposte collegate a una discussione radice):
`let rootId = c.REF_CommentID; while (commentMap.has(rootId) && commentMap.get(rootId).REF_CommentID`
`{ rootId = commentMap.get(rootId).REF_CommentID; }`. In sostanza, Tramite l'ausilio di una mappa (`Map`) e attraverso un ciclo `while`, il codice risale la catena dei commenti fino a trovare il commento padre (la radice). Successivamente inserisce la risposta nell'array `.replies` di quel padre assoluto. Questo approccio evita l'annidamento infinito (callback hell visivo) e mantiene l'interfaccia pulita e leggibile anche su schermi mobili.

Il metodo `parseMarkdown` funge da filtro di sicurezza per la formattazione del testo. Prima di procedere alla conversione della sintassi la funzione esegue l'escape dei caratteri riservati HTML (`<`, `>`, `&`). Questo processo neutralizza l'eventuale iniezione di codice arbitrario (*script injection*), scaricando sul client l'onere della pulizia dei dati prima del rendering dinamico nel DOM.

I metodi `rispondiA` e `scrollToComment` estendono le API native del browser (`scrollIntoView` e `setFocus`) sincronizzandole con il ciclo di vita dei rendering di Angular tramite macro-task (`setTimeout`). Questa implementazione garantisce transizioni fluide dell'interfaccia e l'attivazione automatica delle tastiere software sui sistemi operativi Android e iOS durante le interazioni rapide.

Discussion-modal: definisce l'interfaccia grafica e la logica di controllo adibita alla manipolazione e alla configurazione dei canali di discussione (thread dei commenti) associati agli episodi all'interno della piattaforma NexuStream. La gestione dello stato dei dati fa uso dei *Reactive Forms*. All'atto del caricamento (`ngOnInit`), il sistema computa lo stato operativo tramite il flag `isEditMode`. Qualora il componente operi in modalità di inserimento, vengono applicati dinamicamente vincoli di validazione stringenti sui campi chiave (`REF_EpisodeID` e `type`); in modalità di aggiornamento, tali vincoli vengono deregistrati per proteggere l'immutabilità dei vincoli relazionali del database, esponendo esclusivamente i campi di alterazione temporale (`closeDate`) e di blocco amministrativo (`forceClosed`). Al fine di garantire la perfetta corrispondenza con i tipi di dato nativi dello strato di persistenza SQL (tipologia `DATETIME`), il campo adibito alla data di chiusura integra una validazione basata su espressioni regolari.

All'atto del salvataggio (**save**), la classe invoca il metodo **getRawValue()** in sostituzione della proprietà standard **value**. Questa scelta è indispensabile per bypassare il comportamento nativo del framework, il quale escluderebbe i campi in sola lettura (**readonly**), garantendo il recupero dell'identificativo dell'episodio. Il flusso esegue infine un controllo di sicurezza tramite la funzione **isNaN()**, bloccando programmaticamente la propagazione di richieste asincrone qualora si verificano anomalie nel passaggio dei parametri tra i componenti della vista.

Episode-card: rappresenta l'interfaccia grafica di una Card per i singoli Episodi, inserita all'interno della vista di una stagione o di una serie TV di NexuStream. Essa gestisce lo stato di riproduzione dell'utente (la barra di avanzamento) e ottimizza la gestione degli eventi per evitare bug di navigazione. Lo stato di avanzamento della riproduzione dell'utente è recuperato dal backend. Tramite il costrutto **@if/@else if**, l'interfaccia discrimina tra contenuti parzialmente fruiti e completati. Nel caso di visione parziale, viene applicato un *Property Binding* dinamico sulla proprietà CSS **width** (**[style.width]**), iniettando il valore percentuale a runtime per aggiornare la barra di progresso geometrico sulla miniatura. Per garantire la precisione delle interazioni, i metodi di riproduzione (**onPlay**) e di consultazione dei metadati (**onInfo**) ricevono l'oggetto nativo del DOM **\$event**. Questa configurazione permette la chiamata al metodo **stopPropagation()** nel file di controllo, inibendo il fenomeno del *bubbling* degli eventi e impedendo l'attivazione accidentale della riproduzione video durante la selezione dei dettagli. Il caricamento della sorgente visiva prevede un meccanismo di fallback condizionale che sostituisce le miniature mancanti o corrotte con un asset segnaposto locale (*placeholder*). Inoltre, le metriche temporali grezze espresse in secondi dal database vengono normalizzate inline tramite espressioni matematiche nel template (**Duration / 60**) per ottimizzare l'esperienza di lettura.

La logica di controllo associata alla card dell'episodio è definita all'interno della classe **EpisodeCardComponent**, il cui unico scopo è la formattazione visiva dei dati in ingresso e la propagazione controllata delle interazioni verso i componenti superiori. Mediante l'utilizzo combinato del decoratore **@Output()** e della classe **EventEmitter**, il componente espone verso l'esterno due canali di notifica fortemente tipizzati: **play** e **info**. All'atto del clic, il componente si limita a emettere l'identificativo univoco dell'entità (**EpisodeID**), demandando la risoluzione della richiesta al componente genitore. All'interno dei metodi gestori **onPlay()** e **onInfo()** viene intercettato l'evento del browser. L'invocazione sequenziale di **preventDefault()** e **stopPropagation()** interrompe programmaticamente la catena di propagazione degli eventi (*bubbling*) all'interno dell'albero del DOM.

Language-switcher: si tratta del selettore di lingua ufficiale della piattaforma NexuStream, implementato per supportare la fruizione multilingua dei servizi, adottando una strategia di routing localizzato basata sulla segmentazione degli URL. All'atto dell'inizializzazione del componente (**constructor**), il sistema intercetta l'oggetto nativo **window.location** e isola il primo segmento utile dell'URL para determinare il codice di localizzazione corrente. Un controllo di validazione formale verifica la corrispondenza del costrutto con un array di lingue supportate (**['it', 'en']**), applicando la lingua italiana come configurazione di fallback predefinita in caso di anomalie. Infine il metodo **switchLanguage** sovrascrive l'indice di localizzazione e forza il reindirizzamento del browser tramite l'assegnazione della proprietà **href**;

Mod-discussion-card: a questo componente viene affidata all'interno della dashboard di moderazione, la visualizzazione atomica di ciascuna stanza di discussione associata ai singoli episodi. Il componente comunica con il padre in questo modo:

Riceve dal componente padre un oggetto tipizzato **ModDiscussion** contrassegnato dalla proprietà **{ required: true }**, contenente tutti i metadati relativi alla stanza (tipologia, data di scadenza, stato di chiusura, riferimenti a serie ed episodio);

Esponde due canali basati su **EventEmitter**, denominati rispettivamente **edit** (che emette l'intera istanza della discussione) e **delete** (che propaga esclusivamente l'identificativo numerico **DiscussionID**).

Quando il moderatore interagisce con i pulsanti di gestione o rimozione, il componente non esegue direttamente l'azione, ma emette l'evento verso l'alto (**this.edit.emit(this.disc)** o **this.delete.emit(this.disc.DiscID)**). Sarà compito della pagina contenitore (*Smart Component*) intercettare l'evento, invocare il service di

moderazione remoto ed effettuare il refresh dello stato.

Poiché i titoli del catalogo sono memorizzati nel backend sotto forma di oggetti JSON serializzati, il componente implementa un metodo denominato `parseLang`: l'algoritmo esegue un parsing sicuro all'interno di un costrutto `try-catch` e, in condizioni di normalità, interroga l'oggetto per la lingua del client selezionata (default `'it'`) ed effettua una cascata di fallback sul codice `'en'`.

Il template del componente, tramite espressioni valutate a runtime, applica selettivamente classi stilistiche condizionali sia sull'involucro esterno della card sia sui componenti interni di notifica: l'attributo `[class.is-closed]` modifica l'opacità e l'interattività dell'intera card se la stanza è disattivata (`disc.ForceClosed`), mentre l'elemento `span` muta dinamicamente le proprie proprietà cromatiche (es. verde per lo stato attivo, rosso per quello chiuso), massimizzando l'efficienza visiva dell'interfaccia utente durante le lunghe sessioni di auditing da parte del team di moderazione.

Mod-user-comment: questo componente viene utilizzato all'interno della dashboard di moderazione per visualizzare una singola riga di commento lasciata da un utente e consentire ai moderatori di approvarla o nascondere. Il componente comunica con il padre nel seguente modo:

Riceve l'oggetto del commento da moderare (`comment!:` `ModComment`) contenente tutti i dettagli (testo, data, segnalazioni, ecc.).

Emette un evento verso il componente padre quando si clicca sul tasto di approvazione `toggleApprove`.

Emette un evento quando si clicca sul tasto per nascondere/mostrare il commento `toggleHide`.

La struttura visiva si divide in tre sezioni principali:

1. *Header del Commento:* Mostra la data di pubblicazione formattata (`dd/MM/yyyy HH:mm`). Mostra condizionalmente un badge di segnalazione (`comment-report-badge`) solo se il numero di segnalazioni (`ReportCount`) è maggiore di 0, gestendo anche il plurale ("Segnalazione" vs "Segnalazioni").
2. *Corpo del Commento:* Visualizza il testo vero e proprio contenuto in `comment.CommentText`.
3. *Footer e Azioni di Moderazione:* Mostra il numero di "Mi piace" (`Likes`). Mostra lo stato attuale tramite delle etichette (tag): *Nascosto* (se `isHidden` è vero), *Approvato* (se `isApproved` è vero), o *In attesa* (se non è né nascosto né approvato).

Fornisce due bottoni grafici (`ion-button`) per i moderatori:

Approvazione: Cambia colore (verde/grigio) e icona a seconda che sia già approvato o meno. Cliccando, chiama `onApprove()`.

Visibilità (Nascondi): Consente di oscurare il commento chiamando `onHide()`.

Propic-modal: permette ai catalogatori di caricare nuovi avatar di sistema raggruppati per specifici pacchetti tematici denominati "Bundle". Per la gestione dello stato asincrono e di tracciamento del file (proprietà `selectedFile` e `isUploading`), il componente implementa il paradigma delle *Signals*. L'elemento `<input type="file">` viene escluso dalla vista tramite regole di fogli di stile ed intercettato programmaticamente nella classe TypeScript tramite la direttiva `viewChild`. L'attivazione della finestra di selezione dei file viene delegata ad un pulsante Ionic stilizzato, il quale invoca un evento di click virtuale sull'elemento nativo sottostante tramite l'interfaccia `nativeElement`. Il trasferimento dell'asset verso lo strato backend fa uso dell'oggetto di sistema `FormData`. Questa struttura permette la corretta codifica del payload nel protocollo di trasferimento `multipart/form-data`. Il metodo di invio implementa criteri di sicurezza attiva integrando l'utilità `getAuthHeaders`. Prima di effettuare la chiamata `http.post`, il componente estrae il JSON Web Token precedentemente memorizzato nel `localStorage` del dispositivo e lo inietta all'interno degli header HTTP sotto la chiave standard `Authorization: Bearer <TOKEN>`. Questo permette al middleware di autenticazione del backend di validare l'identità del catalogatore prima di elaborare il file.

Show-card: si tratta di una card multimediale riutilizzabile usata per mostrare le serie e gli episodi all'interno della dashboard dell'applicazione. Le API di Input/Output sono basate su *Signals*. L'obbligatorietà del costrutto `show` garantisce la validazione formale dei dati a tempo di compilazione (*compile-time*

checking), prevenendo eccezioni dovute a record incompleti provenienti dalle API esterne. L'architettura del modulo consente a un'unica istanza di mutare comportamento visivo e logico in base al contesto d'uso della dashboard. Infatti attraverso l'uso dei blocchi condizionali nativi del framework (**@if**), il componente supporta una doppia modalità di visualizzazione:

Stato Portrait (Iconografico): Ottimizzato per le griglie di esplorazione verticali, mappa l'URI della miniatura della locandina (**ThumbnailURI**) elaborandola tramite la pipe di risoluzione dell'host back-end;

Stato Landscape (Cronologico): Configurato per i caroselli orizzontali. Oltre a modificare l'asset visivo in formato panoramico (**BannerURI**), introduce dinamicamente il componente per il tracciamento dello stato di visione. Leggendo il metadato **Progress** iniettato dal server, il sistema calcola la percentuale lineare e aggiorna in tempo reale lo stile CSS del riempimento della barra di avanzamento (**progress-bar-fill**).

Per assicurare la massima accuratezza delle interazioni, tutti i metodi delegati alle azioni utente (**onPlay**, **onInfo**, **onRemove**, **onDismiss**) gestiscono l'oggetto nativo **Event** del DOM. L'invocazione sistematica del metodo **event.stopPropagation()**, interrompe il fenomeno della propagazione degli eventi verso l'alto prevenendo l'attivazione simultanea e conflittuale di più eventi di rotta.

Video-player: gestisce la riproduzione dei flussi adattivi HLS ed estende l'esperienza d'uso con logiche tipiche delle piattaforme di streaming commerciali, integrando la libreria Video.js. Il componente esegue un ciclo sui marcatori temporali dell'episodio (**times**). Se il tempo corrente rientra in una fascia definita, si attivano in sovrapposizione i pulsanti di azione gestiti tramite Angular *Signals*:

intro / recap: Mostra istantaneamente il pulsante “SALTA INTRO” o “SALTA RIASSUNTO”. Al click, il metodo **skipTo()** sposta la testina del player esattamente al secondo finale del marcatore (**EndTime**), ottimizzando i tempi di visione;

credits (Titoli di coda): Attiva un overlay di fine visione. Se l'utente non interagisce entro 5 secondi (**setTimeout**), il sistema invoca autonomamente **triggerNextEpisode()**, avviando la riproduzione dell'episodio successivo (*Autoplay Pattern*). Se l'utente clicca su “GUARDA I TITOLI”, il timer viene annullato (**clearNextEpTimer()**) e il popup nascosto.

Il browser esegue il metodo **.play()** in modo asincrono, restituendo una **Promise**. Se si tenta di distruggere o mettere in pausa il player prima che questa **Promise** si sia risolta, l'applicazione lancia un errore critico. Nel metodo **killPlayer()** è stato inserito un controllo di sincronizzazione garantendo una chiusura pulita. Nel metodo **triggerNextEpisode()**, l'evento di uscita viene spinto nel ciclo successivo del motore JavaScript del browser tramite un micro-ritardo senza il quale Angular distruggerebbe il componente durante l'elaborazione del click o dell'evento nativo **ended** di Video.js. Il metodo **killPlayer()** implementa una procedura di deallocazione a cascata di livello industriale:

Reset Audio: Azzerà il volume e muta il tag nativo per evitare glitch sonori durante lo smantellamento;

Saturazione dello Stream: Rimuove l'attributo **src** e invoca **.load()** per forzare il browser a liberare immediatamente la memoria tampone occupata dall'ultimo frammento video caricato;

Disconnessione Eventi: Invocando **this.player.off()**, si rimuovono tutti i listener, impedendo a funzioni isolate di girare a vuoto in background;

Interruzione Richieste HTTP di Rete (VHS Abort): intercetta l'estensione video di Video.js e annulla istantaneamente qualsiasi download asincrono attivo dei segmenti video successivi, bloccando lo spreco di banda sul server;

Rimozione Finale: Invocando **.dispose()**, Video.js pulisce il DOM e cancella l'istanza globale,

lasciando il puntatore pronto per il Garbage Collector di JavaScript (`this.player = null`).

5.1.1.2 Guards

All'interno della directory `/guards` si trovano le Route Guards del nostro sistema. Ognuno di questi moduli ottiene il token di sessione dal `localStorage`. Le guard `auth-guard` e `guest-guard` verificano se l'utente ha effettuato l'accesso oppure è un guest. Il primo nega l'accesso ad alcune parti dell'applicazione, per esempio la visione di un episodio, se l'utente non è autenticato. Le restanti guard decodificano il token tramite una libreria nativa (`jwtDecodeHelper`): se la decodifica è andata a buon fine e il flag corrispondente (`isAdmin`, `isCat` o `isMod`) del `decodedToken` è impostato al valore 1, allora viene consentito l'accesso alla rotta, altrimenti viene restituito un messaggio di errore e l'utente viene reindirizzato alla pagina `forbidden`.

5.1.1.3 Models

Nella directory `/models` si trovano i modelli dei dati, ovvero le interfacce che rappresentano lo strato formale deputato alla definizione tipizzata dei dati che transitano tra i componenti dell'interfaccia utente (UI) e gli strati di logica di business governati dai Servizi HTTP (`services`).

5.1.1.4 Pages

Le pagine rappresentano le interfacce principali con cui interagiscono gli attori dell'applicazione. Di seguito vengono elencate le pagine di cui è composto `NexuStream` e la relativa logica:

Admin: questa pagina costituisce il modulo di controllo e backoffice riservato agli amministratori per l'auditing e la manipolazione dei privilegi d'accesso degli utenti. Il modulo adotta una griglia di visualizzazione fluida (`admin-grid`) che ottimizza la resa grafica delle informazioni anche in presenza di volumi di dati consistenti. Infatti la pagina implementa un sistema di caricamento asincrono incrementale accoppiato al componente `<ion-infinite-scroll>`. Impostando una dimensione fissa della finestra di fetch (`pageSize = 20`), l'applicazione intercetta lo scostamento verticale dell'utente rispetto al fondo del viewport (soglia fissata a 150px), determinando l'avanzamento incrementale dell'indice di paginazione.

La pagina integra una barra di ricerca contestuale in grado di scandire l'elenco utenze per stringhe testuali (agendo su corrispondenze di username o email). Onde prevenire un sovraccarico di richieste asincrone verso gli endpoint del server, il componente implementa una barriera temporale sul controllo di input: la proprietà `[debounce]="500"` istruisce il client a ritardare la sottomissione dell'evento `onSearch` di 500 millisecondi dall'interruzione della digitazione dell'utente.

Lo stato interno della pagina è governato mediante gli Angular Signals (`signal<AdminUser[]>()`), i quali notificano in tempo reale al DOM i mutamenti occorsi alla collezione a seguito di filtri o azioni di aggiornamento (`toggleRoleDirectly`).

Il template elabora la matrice dati attraverso la direttiva accoppiata `@for`, mappando le card dei profili con classi stilistiche differenziate (es. `.edge-admin`) qualora il soggetto esaminato detenga poteri di amministrazione stabili. La classificazione funzionale dell'utente viene esplicitata visivamente attraverso un sistema di badge condizionali annidati, i quali valutano a runtime i flag booleani restituiti dal record.

Cataloguer: questa pagina costituisce l'infrastruttura di Content Management System (CMS) interna di `NexuStream`, riservata agli utenti con privilegi di catalogazione. La pagina centralizza in un'unica vista logica due macro-aree operative: la gestione del catalogo multimediale ad albero (Anime, Stagioni, Episodi) e l'amministrazione delle immagini per i profili utente (Libreria Avatar). Modificando il valore del marcatore `currentLevel`, il template HTML isola e renderizza selettivamente la griglia di competenza mediante le direttive condizionali `@if`:

Livello shows: Consente il controllo globale sulle serie TV/Anime in archivio, supportando la ricerca asincrona e il caricamento a scorrimento infinito (`ion-infinite-scroll`);

Livello seasons: Vincolato all'identificativo memorizzato `selectedShowId`, mostra la segmentazione delle stagioni associate alla serie selezionata;

Livello episodes: Ancorato al contesto `selectedSeasonId`, espone l'elenco atomico degli episodi, tracciandone metadati complessi quali durata, tracce audio, sottotitoli e marcatori temporali.

Il ripristino della gerarchia superiore avviene in modo sicuro tramite il metodo `MapsBack()`. Per supportare l'internazionalizzazione dell'indice multimediale, la pagina implementa la routine di decodifica `getLangText()` che intercetta i flussi dati estratti dai Signals ed esegue il parsing sicuro del payload. La manipolazione e il popolamento dei record avvengono tramite l'interfacciamento con il servizio `CataloguerService`. I metodi helper `getAddModalConfig` e `getEditModalConfig` isolano il tipo di componente modale da istanziare (`CataloguerShowModalComponent`, `CataloguerSeasonModalComponent` o `CataloguerEpisodeModalComponent`) e calcolano programmaticamente i parametri predittivi da passare in input (es. l'autocompilazione incrementale del numero dell'episodio o della stagione rispetto ai record esistenti). All'atto del salvataggio, il componente adotta logiche asincrone per garantire l'allineamento dello stato dell'applicazione senza richiedere il ricaricamento distruttivo della pagina:

In fase di inserimento (Add): La risposta HTTP del server Express, contenente la chiave primaria auto-incrementale generata dal DBMS, viene catturata dal metodo `updateSignalsAfterAdd()`, il quale ricostruisce l'oggetto tipizzato e lo inietta in testa all'array del Signal di riferimento tramite operatore di sfasamento.

In fase di modifica (Edit): Data la complessità dell'entità episodio, l'applicazione inoltra al backend richieste parallele rese sequenziali tramite `firstValueFrom` per effettuare il patching differenziale delle tabelle di traduzione (Italiano/Inglese) e dei metadati fisici (file multimediali, tracce audio e video, marcatori `EpisodeTime`). Successivamente, muta reattivamente lo stato in memoria RAM mediante operazioni di `.map()` sul Signal, garantendo una UX fluida e priva di latenze di caricamento.

Commutando il segmento attivo (`onTabChange()`) su `propics`, la pagina muta il proprio contesto operativo offrendo un'interfaccia di manutenzione degli asset grafici statici della piattaforma. Sfruttando il metodo `loadPropics`, la pagina interroga gli endpoint di backoffice (`cataloguerService`) ottenendo la lista dei file mappati e raggruppati all'interno di strutture dati denominate `PropicGroup`. Il template esegue il rendering dei file aggregati per pacchetto di appartenenza (*Bundle*), fornendo agli operatori i comandi per l'upload massivo di nuove immagini del profilo o per l'eliminazione distruttiva (`deletePropic`) dei file fisici memorizzati sul file system del server remoto, con notifica di feedback gestita mediante `ToastController`;

Episode: questa pagina rappresenta l'interfaccia a più alta densità computazionale e di interazione, gestendo in modo reattivo flussi video, monitoraggio dello stato di avanzamento, controllo degli accessi e moderazione in tempo reale.

I flussi asincroni estratti da `ActivatedRoute` (relativi all'identificativo dell'episodio) e da `queryParamMap` (relativi ai vincoli gerarchici di serie e stagione) vengono unificati in un unico canale di ascolto mediante l'operatore concorrente `combineLatest`. Questo impianto architetturale permette alla pagina di gestire in modo trasparente il cambio sequenziale di risorsa (es. selezione di un altro episodio dalla sidebar laterale): prima di aggiornare i Signals dello stato e iniettare la nuova URL nel player, il client interzetta la mutazione, invoca in modo sincrono la distruzione controllata del thread del lettore multimediale (`killPlayer`) e memorizza sul database remoto lo stato di avanzamento esatto raggiunto fino a quel momento dall'utente.

Per mitigare il carico sulla rete client-server ed evitare la saturazione dell'endpoint API di interazione (`EpisodeService.interact`), è stata progettata una strategia difensiva di tipo *Debounced Save*:

Salvataggio Ritardato: Agli eventi di transizione non definitivi, come la messa in pausa del flusso video, l'applicazione non istanzia immediatamente una richiesta HTTP. L'esecuzione viene demandata a una routine asincrona governata da un `setTimeout` con finestra temporale fissa di 1000ms. Eventuali micro-variazioni dello stato nello stesso intervallo cancellano e rischedulano il timer, min-

imizzando il throughput dei pacchetti di rete;

Salvataggio Immediato: In corrispondenza di macro-eventi deterministici (completamento del video `handlePlayerEnded`, distruzione del modulo o attivazione degli hook del ciclo di vita della pagina come `ionViewWillLeave`), il parametro di guardia `immediate` commuta su `true`. Il client esegue un flush immediato del payload verso il backend Express, garantendo la consistenza assoluta dello storico di visione dell'utente nel DBMS relazionale.

La sicurezza e la segmentazione dei contenuti a livello di interfaccia sono governate dinamicamente dall'estrazione dello stato di autenticazione archiviato nel modulo `localStorage`. All'atto del bootstrap della pagina, il client verifica la validità del token JWT tramite la routine helper `jwtDecodeHelper`:

Utente Autenticato: Viene eseguito l'unboxing dell'element custom `<app-video-player>`, abilitando l'aggancio dinamico della sorgente streaming protetta e decodificando l'array dei metadati temporali (`EpisodeTimes`) adibiti ai marcatori dei capitoli (intro/outro). Nel caso in cui il payload del token certifichi i privilegi di moderazione (`isMod === 1`), il template abilita condizionalmente i Web Components per la creazione e la rimozione distruttiva delle discussioni d'ascolto.

Utente Anonimo (Guest): Il riproduttore video viene programmaticamente escluso dal DOM. Al suo posto, un blocco condizionale `@else` renderizza una vista di Paywall nativa: viene esposta l'immagine di anteprima dell'episodio elaborata dalla pipe di sanificazione degli indirizzi statici (`BackendUrlPipe`) sovrapposta a un overlay grafico sfocato che inibisce l'interazione e reindirizza l'utente anonimo alla pagina di login.

La pagina implementa un motore di navigazione predittivo per la riproduzione continua. Infatti, all'emissione dell'evento di terminazione del video, il metodo `handleNextEpisode()` analizza la collezione ordinata del Signal `seasonEpisodes`. Qualora non si riscontri un episodio successivo all'interno della medesima stagione, interviene la routine `checkAndNavigateToNextSeason()`, la quale interroga asincronamente la gerarchia parentale della serie: se mappata, estrae la stagione successiva, effettua il fetch del rispettivo primo episodio e forza il reindirizzamento della rotta sul client, gestendo la transizione visiva mediante notifiche asincrone fornite dal `ToastController`.

La pagina inoltre funge da mediatore tra il riproduttore multimediale e la sezione commenti. All'attivarsi dell'evento `timestampClick` generato dal sottocomponente `<app-comments>`, la pagina intercetta i secondi corrispondenti e pilota via hardware il lettore video (`this.videoPlayerComponent.seekTo(seconds)`), riposizionando poi fluidamente lo scroll dello schermo verso l'alto tramite le API native di `IonContent`;

Favourites: la pagina `FavouritesPage` (esposta all'utente con il titolo "La Mia Lista") costituisce lo spazio di persistenza e consultazione delle serie contrassegnate come preferiti dall'utente autenticato. La pagina inietta via Dependency Injection il modulo `FavoritesService` per interloquire con gli endpoint del server Express. Il rendering visivo della griglia multimediale (`favorites-grid`) non viene implementato in modo monolitico, ma riutilizza ricorsivamente il componente isolato `<app-show-card>` mediante la direttiva nativa `@for` applicata al Signal `favorites`.

La pagina calcola e distribuisce i dati verso la card (proprietà `[show]`) e ne abilita condizionalmente i comandi di cancellazione (`[showDelete]="true"`). Le interazioni dell'operatore non vengono gestite autonomamente dalla card, ma emesse verso l'alto tramite tre canali di `@Output` (`play`, `info`, `remove`). Per eliminare la percezione della latenza di rete durante la rimozione dei contenuti dalla lista, il metodo `removeFromFavorites` adotta un approccio algoritmico di tipo ottimistico. All'attivazione del comando di rimozione, il componente effettua una copia speculare dello stato corrente all'interno della variabile locale `oldFavs` e muta istantaneamente il Signal locale tramite un filtro booleano sull'identificativo `showId`. Questo aggiornamento immediato del DOM simula una reattività istantanea dell'applicazione.

La tolleranza ai guasti è garantita dalla sottoscrizione all'Observable: nell'evenienza di un fallimento di rete o di un errore del database remoto, il blocco `error` intercetta l'eccezione, innesca una routine di Rollback dello stato ripristinando il Signal al valore di backup (`this.favorites.set(oldFavs)`) e allerta l'utente mediante un'interfaccia di avviso asincrona coordinata dal `ToastController`.

Il ciclo di vita visivo della schermata è vincolato al monitoraggio dello stato di caricamento tramite il marcatore reattivo `isLoading`. Durante la fase di fetch dei dati operata all'interno dell'hook `ionViewWillEnter`, il client esclude i componenti di computazione e mostra un overlay di alta precisione standardizzato (`ion-spinner`). Al completamento della transazione, qualora la collezione restituisca un array vuoto, la direttiva condizionale `@else` isola il layout e istanzia un blocco di *Empty State*. Quest'ultimo, supportato dall'icona di sistema `heart-dislike-outline`, fornisce all'utente un feedback grafico chiaro e contestualizzato, evitando la visualizzazione di schermate bianche prive di significato;

Forbidden: riveste un ruolo fondamentale nell'infrastruttura di sicurezza e controllo degli accessi dell'applicazione client. Agisce come punto di atterraggio definitivo per le Guardie di navigazione di Angular ogni volta che un utente tenta un'escalation di privilegi non autorizzata. Al completamento del ciclo visivo, l'utente è guidato verso una transizione di ripristino controllata: l'interazione con il comando grafico di ritorno aziona il metodo `goToHome()`, il quale inietta via Dependency Injection il modulo `Router` per de-escalare in sicurezza la navigazione riportando il viewport sul percorso radice protetto dell'applicazione (`/tabs/home`);

Forgot-password: questa page rappresenta la schermata di recupero account tramite password reset. L'acquisizione dei dati è governata dal modulo *Reactive Forms* di Angular, iniettato mediante il servizio `FormBuilder`. L'applicazione vincola il ciclo di vita del form all'applicazione concorrente di due validatori atomici: la presenza obbligatoria del dato (`Validators.required`) e la conformità sintattica della stringa alle espressioni regolari dello standard e-mail (`Validators.email`). La sottomissione del payload è vincolata allo stato del form: il pulsante grafico di invio nel template HTML implementa un property binding condizionale che disabilita programmaticamente l'interazione (`[disabled]`) qualora i criteri sintattici non siano interamente soddisfatti o sia già in corso un thread di comunicazione asincrono (`isLoading`). All'attivazione dell'evento di sottomissione `onSubmit()`, il componente esegue una routine difensiva di sanificazione sul flusso dati immesso dall'utente prima di strutturare la chiamata di rete. Attraverso l'invocazione del metodo `trim()`, il client rimuove gli spazi bianchi posizionati in testa o in coda alla stringa. Nell'eventualità di un'anomalia, l'esecuzione viene abortita e il client allerta l'operatore mediante un feedback visivo istanziato dal `ToastController`;

Home: La pagina `HomePage` costituisce il punto di ingresso primario (Dashboard) e il centro nevralgico di distribuzione dei contenuti all'interno dell'applicazione client `NexuStream`. La schermata organizza i contenuti multimediali in strutture sistematiche parallele e caroselli a scorrimento.

Per garantire tempi di caricamento ottimali ed eliminare i fenomeni di sfarfallio visivo causati da caricamenti asincroni frammentati, il ciclo di vita del componente centralizza il fetching dei dati all'interno dell'hook `ionViewWillEnter`. La pagina interroga in parallelo gli endpoint del servizio `HomeService` strutturando le richieste tramite l'operatore di combinazione concorrente `forkJoin`. Questo consente la risoluzione simultanea di cinque canali di dati indipendenti: la selezione dei contenuti in evidenza (*Hero List*), le ultime release immesse a catalogo, le metriche di popolarità basate sulle visualizzazioni (`mostViewed`) e sugli apprezzamenti degli utenti (`mostLiked`). Inoltre, qualora l'applicazione rilevi la presenza di un token JWT valido per una sessione attiva, la pipeline aggancia condizionalmente il feed personalizzato di ripresa della visione ("Continua a guardare").

Nella sezione superiore dell'interfaccia, i contenuti di manifesto vengono valorizzati attraverso un componente di rotazione automatica (*Banner Hero*). L'avanzamento sequenziale dell'indice attivo è regolato da una routine temporizzata operante a intervalli di 5000ms, istanziata dal metodo `startHeroTimer()`.

La navigazione trasversale delle categorie multimediali sfrutta le API native del browser combinate con i decorator `@ViewChild`. La classe acquisisce i riferimenti diretti agli elementi del DOM (`#recentScroll`, `#mostViewedScroll`, `#mostLikedScroll`) incapsulando il puntatore hardware all'interno di istanze `ElementRef`. All'attivazione dei comandi grafici a freccia, il metodo `scrollRow` calcola lo scostamento geometrico in pixel a seconda della direzione passata in input (*left* o *right*) e pilota direttamente la proprietà di scorrimento del contenitore mediante l'istruzione `scrollBy` impostando un comportamento di transizione fluido (`behavior: 'smooth'`).

Il modulo estende il paradigma degli aggiornamenti ottimistici all'interno della gestione della coda di

visione personale dell'utente (`continueWatching`). Se un utente decide di escludere un contenuto dal proprio feed tramite l'interfaccia di opzioni, il metodo `removeFromContinueWatching` esegue preventivamente un filtraggio sul Signal reattivo locale, rimuovendo visivamente l'elemento dal DOM in tempo reale. Successivamente, l'applicazione inoltra in background il payload transazionale verso l'endpoint REST (`updateEpisodeInteraction`) configurando il marcatore `isDropped`: 1. Qualora la comunicazione HTTP fallisca per anomalie di rete, il blocco di intercettazione dell'errore esegue un rollback dello stato ripristinando la collezione salvata nel backup `oldList` e informando l'utente tramite un avviso pop-up emesso dal `ToastController`;

Login: costituisce il varco di accesso protetto per l'autenticazione delle sessioni utente all'interno di NexuStream. Questo modulo centralizza le logiche di verifica sintattica delle credenziali, la sottomissione dei payload verso i service, l'estrazione dinamica dei profili autorizzativi e l'allineamento del router client in base ai dizionari di lingua attivi. L'acquisizione dei dati immessi dall'operatore si affida al modulo *Reactive Forms*, definendo un'istanza di `FormGroup` che incapsula i controlli per i campi email e password. L'integrità del payload è protetta a livello di interfaccia mediante l'applicazione concorrente di validatori sincroni. Il template HTML adotta un approccio dichiarativo guidato dallo stato logico: il comando grafico di sottomissione (`type="submit"`) mappa un vincolo di disabilitazione hardware (`[disabled]`) ancorato alla proprietà di invalidità del form (`loginForm.invalid`), azzerando l'inoltro di richieste HTTP strutturalmente incomplete verso il server Express. Sotto il profilo visivo, il consumo del web component nativo di Ionic `<ion-input-password-toggle>` garantisce la mascheratura sicura dei caratteri della chiave d'accesso.

A seguito della risoluzione positiva dell'Observable associato al metodo `authService.login`, l'applicazione riceve dal server un payload contenente il token JWT e l'oggetto utente profilato. Data la potenziale variabilità delle risposte del database relazionale, il componente implementa la routine privata difensiva `buildRolesArray()`. Questo metodo esegue un'ispezione sulle proprietà booleane dell'entità utente (esaminando in parallelo i campi `isAdmin`, `is_admin`, `isMod`, `is_mod`, `isCataloguer`, `is_cataloguer`). Le stringhe corrispondenti ai ruoli attivi vengono inserite all'interno di un array di stringhe tipizzato. Qualora il payload non espliciti alcun privilegio amministrativo, la funzione attua una strategia di fallback assegnando programmaticamente il ruolo base `['user']`. Tale operazione previene l'insorgenza di crash applicativi causati da valori undefined e normalizza lo stato della sessione prima del salvataggio nel `localStorage` del dispositivo.

Poiché il framework distribuisce l'applicazione in sotto-percorsi URL isolati per ciascuna lingua (`/it/` o `/en/`), il metodo di login non si limita a invocare il metodo di navigazione standard del router, ma analizza preventivamente il contesto geografico tramite il servizio `LanguageService`. Se l'applicazione rileva che la lingua attiva memorizzata nel client (`appLang`) non coincide con il prefisso attualmente presente nell'indirizzo URL del browser, il componente forza una riassegnazione hardware del viewport tramite l'istruzione `"window.location.href = window.location.origin + '/' + appLang + targetRoute"`. Questa ricarica mirata garantisce che l'utente venga rediretto sulla dashboard corretta (`/tabs/home`). Le eccezioni o i fallimenti di autenticazione vengono catturati dal blocco `error`, il quale inibisce l'accesso e notifica l'errore sintattico mediante stringhe tradotte dinamicamente a runtime tramite il tag `$localize` integrato nel `ToastController`;

Mod: questa pagina rappresenta il pannello di backoffice dedicato alla moderazione di commenti, discussioni e utenze. Attraverso il marcatore reattivo `currentTab`, l'applicazione isola i comportamenti visivi e i criteri di filtraggio del template HTML mediante direttive condizionali `@if`:

Tab discussions: Abilita condizionalmente i comandi per la creazione di nuovi spazi di confronto (`openCreateModal`) e attiva i filtri per ordinare le discussioni in base ai flag di approvazione, visibilità globale o segnalazioni.

Tab users: Riconfigura la barra superiore per ospitare un componente di ricerca asincrona testuale (`ion-searchbar`) focalizzato sulla scansione dei nickname degli iscritti della piattaforma.

Questa architettura flessibile si riflette nel metodo `loadTabFields()`, il quale parzializza il carico computazionale interfacciandosi con il modulo `ModService`. Il client esegue le richieste HTTP in modo

selettivo verso gli endpoint specifici solo per la tab visualizzata.

Per evitare la saturazione del thread principal di rendering causata dall'inserimento di un numero indefinito di elementi nel DOM, la pagina adotta una struttura gerarchica governata dai web components di Ionic `<ion-accordion-group>` e `<ion-accordion>`. La collezione principale viene scorsa tramite la direttiva reattiva `@for` applicata al Signal `usersList`. Ciascun elemento dell'accordion funge da contenitore autonomo per l'anagrafica dell'utente. I commenti ad esso collegati vengono iniettati ricorsivamente nel sotto-componente specializzato `<app-mod-user-comment>`.

Il disaccoppiamento tra lo stato parentale e quello di presentazione è garantito dall'adozione di un pattern ad eventi (*Event-Driven Architecture*): le azioni disciplinari atomiche sui singoli testi (approvazione o oscuramento) non alterano direttamente il record, ma vengono emesse dal componente figlio verso l'alto tramite canali di `@Output` (`toggleApprove`, `toggleHide`). La pagina `ModPage` intercetta queste emissioni, esegue il patching remoto sul server Express e aggiorna lo stato in memoria mutando in modo controllato la collezione del Signal parentale.

Il controllo disciplinare e l'enforcing delle policy di utilizzo della piattaforma sono orchestrati dal metodo `openUserActions()`, il quale mappa le risposte dell'operatore interfacciandosi con il servizio `ActionSheetController` di Ionic. All'attivazione del comando sulle opzioni utente, il componente esegue una verifica predittiva sui metadati dello stato in esame (quali la presenza di ban attivi o la revoca del diritto di scrittura `canComment`) e costruisce dinamicamente a runtime un menu ad albero di pulsanti condizionali. Oltre al blocco o allo sblocco permanente dei privilegi di scrittura, l'interfaccia supporta logiche di sospensione temporale avanzate, configurando opzioni per l'allontanamento coatto dalla piattaforma per una durata fissa (24 ore o 7 giorni). Il client calcola programmaticamente il timestamp futuro e lo inoltra all'endpoint dedicato del backend Express (`executeBan`). All'atto della risoluzione positiva dell'operazione, la pagina attiva la routine privata `refreshSingleUserStatus()`, la quale esegue un fetch mirato di sincronizzazione in background per aggiornare in modo non distruttivo i flag di stato della sola riga modificata, notificando il successo dell'azione transazionale tramite messaggi localizzati istanziati dal `ToastController`;

Notfound: questa pagina rappresenta la schermata di errore HTTP 404 (Pagina Non Trovata). Il modulo funge da barriera protettiva per l'usabilità, impedendo che errori di digitazione dell'URL o collegamenti ipertestuali interrotti (*broken links*) causino schermate bianche prive di riscontro semantico. Il componente viene agganciato come nodo terminale della pipeline mediante la sintassi a doppia wildcard (`**`). Questo posizionamento strategico fa sì che il framework, qualora fallisca la risoluzione sequenziale di tutte le rotte prefigurate (siano esse pubbliche o protette da guardie di sicurezza), deleghi automaticamente il rendering a questo modulo. Il template implementa un pulsante ad attivazione asincrona ancorato al metodo `goToHome()`;

Register: questa pagina è alla creazione di nuove utenze standard all'interno di NexuStream. Il modulo organizza l'acquisizione dei dati mediante un layout asimmetrico sdoppiato (*Split Layout*), delegando la colonna sinistra alla raccolta dei flussi testuali e lo spazio di destra alla personalizzazione grafica dell'identità visiva dell'account.

La gestione dei dati e il controllo di integrità dell'input utente sono governati dal pattern *Reactive Forms* di Angular. L'istanza principale di `FormGroup` coordina e convalida in tempo reale lo stato dei controlli atomici collegati ai campi email, password e username. Il client impone vincoli dimensionali stringenti sul DOM, vincolando l'accettazione del nome utente all'intervallo esplicito compreso tra 3 e 24 caratteri totali via `Validators.minLength(3)` e `Validators.maxLength(24)`.

Il pulsante grafico di sottomissione posizionato all'interno del form implementa un property binding che interdice programmaticamente l'interazione hardware dell'utente (`[disabled]`) qualora lo stato di validità complessivo del form non risulti interamente verificato (`registerForm.invalid`), bloccando sul nascere l'inoltro di payload incompleti o non conformi.

Una caratteristica di rilievo del modulo risiede nell'integrazione reattiva tra il form testuale e il sotto-sistema di selezione dell'avatar di profilo. L'applicazione non impiega un input di caricamento file tradizionale, ma si appoggia a un catalogo centralizzato di risorse grafiche pre-approvate. Lo

stato dell'immagine correntemente selezionata è incapsulato all'interno di un Angular Signal tipizzato (**selected**), il quale notifica i mutamenti direttamente al motore di rendering per aggiornare la visualizzazione sferica del layout. All'attivazione del comando grafico di modifica, il metodo **triggerImageSelection()** apre una pipeline asincrona interfacciandosi con il servizio **ModalController** di Ionic: questa architettura abilita il passaggio sicuro dei metadati dal componente parentale verso l'istanza figlia **AvatarPickerModalComponent**. Al completamento della selezione da parte dell'utente, l'applicazione intercetta la chiusura del dialogo mediante l'hook asincrono **onDidDismiss()**. La routine estrae in sicurezza il payload di ritorno (**data.selectedAvatar**) e invoca il metodo **set()** per aggiornare in modo non distruttivo il Signal parentale, il quale provvede a sua volta a sincronizzare il valore finale all'interno della form reattiva prima della sottomissione globale. Al completamento del processo di inserimento, l'Observable agganciato al metodo **authService.register** instrada l'utente verso la vista di login controllata, notificando il successo dell'operazione tramite il **ToastController**;

Reset-password: questo modulo è dedicato alla reimpostazione della password tramite token di sessione sicuro. All'atto dell'inizializzazione del componente all'interno dell'hook del ciclo di vita **ngOnInit**, la pagina intercetta il contesto di navigazione inserendo via Dependency Injection il servizio **ActivatedRoute**. La classe esegue una cattura sincrona e atomica dei parametri dell'URL sfruttando le API dello snapshot nativo: `"ngOnInit() { this.token = this.route.snapshot.queryParams['token']; }"`. Questo parametro stringa costituisce il token JWT temporaneo generato dal server Express. Il metodo di sottomissione **onSubmit()** effettua una verifica predittiva su tale stringa: qualora il client rilevi la totale assenza o la malformazione del token nell'URL, interrompe immediatamente la pipeline di invio e notifica un errore di sessione non valida tramite il **ToastController**, impedendo la generazione di chiamate API fallimentari verso il server di backend.

La solida logica di convalida si affida alla strutturazione di un *Reactive Form* regolato da un custom validator accoppiato. Mentre i campi **newPassword** e **confirmPassword** applicano singolarmente i vincoli sintattici obbligatori tramite la classe **Validators**, la coerenza transazionale è blindata a livello di intero **FormGroup** mediante la routine delegata **passwordMatchValidator**. Questa funzione esegue una comparazione binaria tra le due stringhe in tempo reale. In caso di divergenza, inserisce la firma di errore `{ mismatch: true }` nella radice logica del form. Il template HTML consuma reattivamente questa informazione mediante una direttiva condizionale **@if**: la notifica visuale di errore viene mostrata dinamizzandola con l'attributo **touched**, comparando unicamente quando l'utente ha terminato la digitazione nel secondo campo, garantendo un'interazione fluida e priva di falsi positivi grafici.

Prima dell'inoltro definitivo del payload verso l'endpoint REST **authService.resetPassword**, il componente attua una strategia di difesa stringente eliminando le vulnerabilità legate all'inserimento di caratteri invisibili. La stringa viene ripulita tramite l'istruzione nativa **.trim()**; se il risultato determina una stringa vuota, il client blocca la richiesta sul nascere e solleva un'eccezione grafica. Se i controlli vengono superati, il payload composto dal token e dalla nuova chiave sanificata viene trasmesso in background. In caso di risoluzione positiva, l'utente viene reindirizzato in totale sicurezza verso la schermata di login (**/login**) per l'effettuazione del primo accesso con le nuove credenziali aggiornate.

Search: La pagina **SearchPage** implementa l'architettura del motore di ricerca centralizzato della piattaforma NexuStream. Sotto il profilo funzionale, il modulo fornisce agli utenti uno strumento di scansione dinamico in grado di indicizzare e setacciare l'intero catalogo multimediale mediante combinazioni incrociate di stringhe testuali libere (focalizzate su titoli e descrizioni) e filtri tassonomici basati sul genere della serie.

Il componente di ricerca nativo **<ion-searchbar>** è stato configurato introducendo una barriera temporale controllata. L'attributo **debounce="400"** agisce come un temporizzatore hardware all'interno del ciclo degli eventi: l'evento logico **onSearchChange** viene emesso e preso in carico dalla classe TypeScript solo qualora l'utente interrompa la digitazione sulla tastiera per un intervallo minimo di 400 millisecondi. Lo stato di ricerca viene parzializzato in tre nodi reattivi indipendenti: **searchQuery** (stringa di ricerca attiva), **selectedGenre** (ID del genere selezionato o null in caso di visualizzazione estesa) e **searchResults** (la collezione grezza estratta inizialmente dal server).

L'intersezione combinatoria dei criteri è interamente affidata all'algoritmo racchiuso nel Signal computato **filteredResults**: il motore di rendering di Angular ricalcola la griglia dei risultati solo quando

uno dei Signals sottostanti muta effettivamente il proprio valore. Qualora l'utente selezioni ripetutamente lo stesso genere o digiti spazi vuoti, il client blocca l'elaborazione.

Nel template HTML, l'albero dei generi viene renderizzato in una barra di scorrimento orizzontale composta da componenti `<ion-chip>`. Ciascun chip applica dinamicamente una classe CSS condizionale `[class.active-chip]` confrontando l'ID corrente con lo stato reattivo, offrendo all'utente un feedback visivo immediato sul filtro applicato.

La disposizione grafica prevede inoltre una gestione rigorosa degli stati transitori e difensivi dell'applicazione: durante le fasi di caricamento di rete, un indicatore asincrono `<ion-spinner>` sostituisce la visualizzazione dei contenuti per evitare disorientamenti visuali. Qualora la combinazione filtri-testo non produca alcuna corrispondenza all'interno della matrice di dati locale, la direttiva `@if` intercetta l'anomalia e istanzia un blocco grafico dedicato (`div.no-results`), esplicitando l'assenza di record mediante messaggi iper-localizzati tramite i marcatori nativi `!18n` del compilatore. Una volta identificata la risorsa corretta, la sottomissione dell'evento sulla card invoca il metodo `openSeriesInfo()`, delegando al modulo Router l'instradamento controllato verso la pagina di dettaglio del contenuto (`/shows/:id`);

Settings: questa pagina rappresenta il pannello di configurazione del profilo utente, gestione della sicurezza e localizzazione internazionale dell'applicazione. L'interfaccia è strutturata mediante raggruppamenti logici nativi (`ion-item-group`) che separano nettamente le informazioni personali dalle impostazioni applicative e dai comandi di chiusura sessione.

Il caricamento delle informazioni anagrafiche del profilo avviene all'interno del ciclo di vita del componente. Il sistema sfrutta lo stato asincrono monitorato dalla variabile `isLoading` para mostrare un indicatore visivo di attesa (`<ion-spinner>`) qualora il recupero dei dati di sessione richieda un tempo di latenza di rete superiore alla norma.

La selezione di una nuova immagine del profilo (*propic*) non avviene tramite un input di tipo file tradizionale, ma delega l'azione al componente specializzato `AvatarPickerModalComponent`. All'attivazione del metodo `openAvatarSelector()`, il sistema istanzia la modale a runtime: l'interfaccia utente aggiorna immediatamente il Signal locale `currentAvatar` per offrire un feedback visivo istantaneo, mentre la pipeline asincrona sottomette la modifica al backend tramite il servizio `settingsService.changePropic`, gestendo in modo difensivo eventuali anomalie di rete tramite notifiche toast localizzate.

L'attivazione dell'azione `openModifyPasswordModal()` si interfaccia con il modulo `ChangePasswordModalComponent`. Questo approccio garantisce che i form reattivi di convalida delle credenziali, le logiche di controllo sulla complessità dei caratteri e i relativi gestori di errore rimangano confinati all'interno di uno scope isolato, il quale viene allocato in memoria solo su esplicita richiesta dell'utente e rimosso dal DOM non appena la modale viene congedata (`onDidDismiss`).

Per rispondere ai requisiti di internazionalizzazione della piattaforma NexuStream, il componente integra un sistema di commutazione della lingua in tempo reale. Tramite il selettore `<ion-select>`, agganciato alla proprietà reattiva del `LanguageService`, l'utente può modificare la lingua corrente dell'applicazione (es. Italiano o Inglese). Infine, la sicurezza del ciclo di sessione è presidiata dall'azione `logout()`, la quale invoca in modo centralizzato il metodo `confirmLogout()` del modulo di autenticazione para invalidare i token memorizzati nel client e reindirizzare in sicurezza l'utente alla schermata di login;

Shows: questo è il modulo deputato alla visualizzazione dei dettagli di una serie, alla gestione asincrona delle stagioni correlative e alla manipolazione reattiva dello stato dei preferiti.

Il modulo organizza l'esperienza utente dividendo nettamente l'interfaccia in due macro-sezioni: un banner d'impatto visivo (*Hero Banner*) posizionato in testa alla pagina con un overlay a gradiente dinamico, e un'area inferiore asincrona dedicata alla consultazione tassonomica delle stagioni e alla sottomissione dei singoli episodi.

L'acquisizione dei metadati della risorsa è governata all'interno dell'hook del ciclo di vita `ngOnInit` inserendo una pipeline concorrente parallela basata sull'operatore `forkJoin`. Questo approccio mitiga

il fenomeno dei caricamenti seriali a cascata. Lo stato della vista è interamente modellato mediante Angular Signals (`show`, `seasons`, `episodes`, `selectedSeasonId`), i quali notificano i mutamenti direttamente ai nodi del DOM interessati, escludendo cicli di ricalcolo globali ed ottimizzando le performance di rendering.

Il sistema implementa una strategia di caricamento pigro guidata dalle interazioni sul componente di selezione `<ion-select>`. Al variare della stagione selezionata dall'utente, viene intercettato l'evento logico `onSeasonChange($event)`, il quale provvede ad aggiornare il Signal della chiave primaria e ad innescare una richiesta HTTP isolata ed esplicita per la mappatura dei soli episodi subordinati a quel determinato identificativo. Durante questa fase transitoria, una variabile booleana di controllo attiva localmente un indicatore `<ion-spinner>` confinato all'interno del contenitore della griglia (`.episodes-list`), isolando l'attesa di rete senza bloccare o congelare la consultazione delle informazioni primarie della serie (quali titolo, descrizione globale o banner).

Una caratteristica di rilievo del componente risiede nella gestione transazionale dello stato dei preferiti tramite l'azione `toggleFavorite()`. La pagina applica un pattern di *Aggiornamento Ottimistico*. Nel momento in cui viene premuto il pulsante grafico del cuore, l'applicazione inverte immediatamente lo stato logico locale della proprietà `isFavorited` all'interno del Signal parentale e aggiorna la veste grafica della card sul DOM. Solo successivamente viene staccata la chiamata asincrona verso il servizio `showsService.toggleFavoriteStatus`. Qualora la transazione remota fallisca a causa di problemi di connettività o timeout del server, la pipeline intercetta l'eccezione nel blocco di errore, effettua un ripristino conservativo (*Rollback*) riportando la variabile allo stato binario originario ed emette un avviso visuale mediante il `ToastController`, garantendo consistenza atomica tra lo stato real del backend e la rappresentazione visuale del client.

5.1.1.5 Services

Nella directory `/services` si trovano i servizi che si occupano della gestione della logica di business, dello stato dell'applicazione e del disaccoppiamento dei dati dalle interfacce grafiche. Di seguito vengono elencati tutti i servizi e le relative funzionalità:

Admin: servizio per le operazioni di amministrazione del sistema. Gestisce il fetch della lista utenti e l'assegnazione dei ruoli;

Auth: servizio globale per la gestione dell'autenticazione utente. Si occupa di login, registrazione e recupero password;

Avatar-picker: servizio per la gestione dei propic;

Cataloguer: servizio dedicato alle operazioni CRUD (Create, Read, Update, Delete) e di caricamento media del Catalogatore;

Comment: servizio per la gestione dei commenti;

Episode: servizio per la gestione di un determinato episodio. Si occupa di recuperare un determinato episodio, gli altri episodi della stagione corrente, le discussioni relative a un episodio, le stagioni dello show e le interazioni dell'utente con l'episodio;

Favorites: servizio per la gestione dei preferiti dell'utente. Si occupa di recuperare la lista salvata e di gestire le interazioni (aggiunta/rimozione);

Home: servizio globale per la gestione dei contenuti della dashboard principale (Home). Fornisce i feed di serie più viste, più piaciute e la cronologia di visione;

Language: si occupa della gestione dello stato della lingua dell'applicazione, delle tracce audio, dei sottotitoli e della sincronizzazione persistente sia locale che remota;

Mod: servizio dedicato alle operazioni svolte dall'moderatore, ovvero la gestione delle discussioni, degli utenti e del ban;

Search: servizio dedicato alle funzionalità di ricerca globale e filtraggio per genere;

Settings: servizio per la gestione delle impostazioni dell'utente (cambio immagine profilo e password);

Show: servizio per la gestione dei dettagli delle serie TV, delle stagioni e dei relativi episodi.

5.1.1.6 App.component

Il file `app.component.ts` costituisce l'entry point logico e la radice (*Root Component*) dell'intera interfaccia utente della piattaforma NexuStream. All'avvio dell'applicazione, l'istanza esegue immediatamente una routine denominata `checkGlobalLanguageRedirect()`. Questa funzione si occupa di coordinare la persistenza dello stato di localizzazione dell'utente e la coerenza del routing:

1. *Ispezione del Local Storage:* Il componente interroga la memoria del browser cercando la chiave `user_language_preferences`. In assenza di preferenze salvate (es. primo accesso in assoluto), l'applicazione procede con il flusso standard senza forzature.
2. *Rilevamento Linguistico dell'URL:* Se le preferenze esistono, viene estratto il codice della lingua target (es. `'en'` o `'it'`). Parallelamente, tramite l'oggetto `window.location.pathname`, viene analizzato il primo segmento dell'URL attuale para determinare il contesto linguistico di esecuzione corrente.
3. *Dirottamento Programmatico (Forced Redirect):* Nel caso in cui venga rilevata una discrepanza tra la lingua impostata dall'utente e quella espressa dall'URL corrente (ad esempio, se l'utente ha configurato l'inglese ma l'URL punta a `/it/`), il componente interviene manipolando la stringa dell'URL tramite un'operazione di rimpiazzo sintattico. L'utente viene immediatamente reindirizzato alla versione localizzata corretta (`/$targetLang/`), garantendo un'esperienza fluida ed evitando disallineamenti o sfarfallii grafici tra la lingua dell'interfaccia e le preferenze salvate.

La scelta di delegare questo controllo ad `AppComponent` garantisce che il controllo sui moduli di internazionalizzazione avvenga al livello più alto della gerarchia.

5.1.1.7 Locale

Al fine di garantire la massima accessibilità e una distribuzione geografica scalabile della piattaforma NexuStream, lo sviluppo dell'interfaccia utente è stato strutturato adottando le metodologie di Internazionalizzazione (i18n) e Localizzazione (l10n) native del framework Angular. L'architettura fa uso della direttiva `i18n` (e dei relativi attributi derivati come `i18n-placeholder` o `i18n-errorText`), agendo come marcatore di metadati a compilazione statica. In fase di compilazione, il toolchain di Angular estrae tali nodi testuali e genera i dizionari di corrispondenza in formato standardizzato XLIFF (`messages.xlf` e `messages.en.xlf`), collocati nella directory `/locale`.

5.2 Server (Backend)

Mentre il Client (Frontend) si occupa di mostrare un'interfaccia accattivante e fluida all'utente, il Server (Backend) è il motore invisibile che gestisce la logica di business, elabora le query al database SQL, autentica gli utenti tramite token crittografati e protegge le rotte sensibili da accessi non autorizzati.

Nei paragrafi successivi saranno descritti nel dettaglio le parti che compongono il backend.

5.2.1 Server.js

Questo file rappresenta il punto d'ingresso principale del server assolvendo ai compiti di orchestrazione della rete, inizializzazione dei middleware globali, routing stratificato e automazione del database. Tramite il modulo `dotenv` vengono caricati i dati parametrici dal file `.env`, che contiene anche la configurazione di mailtrap utilizzato per l'invio di email di recupero. La documentazione del backend, tramite l'integrazione di Swagger, viene generata automaticamente all'indirizzo `/api-docs`. L'integrazione mappa nativamente lo schema di sicurezza statistica `BearerAuth` (basato su JSON Web Token), permettendo la simulazione

e l'auditing dei flussi di richiesta anche per le rotte protette da privilegi amministrativi, moderativi e di catalogazione.

Il flusso delle richieste HTTP attraversa una serie di filtri globali sequenziali. I criteri di interoperabilità tra domini differenti sono garantiti dall'abilitazione delle policy CORS (*Cross-Origin Resource Sharing*), consentendo il transito sicuro delle richieste asincrone originate dal client Angular. Il parsing automatico dei payload in ingresso è affidato al middleware nativo `express.json()`, mentre la localizzazione dei dati e dei metadati è gestita a monte dal modulo custom `detectLanguage`. `express.static` consente di mappare la cartella fisica "public" del server sull'indirizzo virtuale `/static`, meccanismo che permette alla pipe backendUrl del frontend di scaricare gli avatar degli utenti o le miniature dei video tramite richieste HTTP standard.

Per ottimizzare le procedure di test e sviluppo, l'avvio del server attiva una pipeline sincrona per la gestione del database relazionale. Il ciclo prevede il reset delle tabelle esistenti (`resetDb`), l'esecuzione degli script DDL per l'inizializzazione degli schemi relazionali (`initDb`) e il successivo popolamento tramite dati mock integrati (`populateDb`), assicurando la stabilità e la coerenza dell'ambiente operativo.

5.2.2 Routes

All'interno della directory `/routes` troviamo tutti gli endpoint di NexuStream. L'accesso agli endpoint è regolato nativamente da un sistema di controllo degli accessi basato sui ruoli, implementato mediante la composizione di middleware operanti a livello di trasporto. Gli endpoint si strutturano in tre macro-livelli di sicurezza:

Endpoint Pubblici: Gateway d'ingresso che non richiedono l'invio di credenziali di sessione. Comprendono le rotte di registrazione (`register`), autenticazione (`login`) scelta dell'avatar e i servizi di consultazione di base della libreria media (`home`, `genre`, `search`, `season`).

Endpoint Utente: Rotte dedicate alla fruizione dei servizi core della piattaforma, accessibili esclusivamente previa verifica di un JSON Web Token valido. Includono la gestione del profilo utente (`user`, `propic`) e l'interazione dinamica con il catalogo multimediale (`show`), la quale comprende la visualizzazione degli episodi (`episode`) e la partecipazione ai thread dei commenti (`comments`, `discussion`). La rotta `/show` è accessibile anche senza un token di autorizzazione, ma in quel caso l'utente non può aggiungere ai preferiti lo show in oggetto.

Endpoint di Staff: Rappresentano il perimetro più critico del backend, suddiviso internamente per mansioni specifiche (`adminPage`, `cataloguerPage`, `modPage`, `upload`) al fine di garantire il principio del privilegio minimo. L'accesso è intercettato da middleware dedicati che verificano i claim del JWT.

L'ultimo endpoint, che non rientra nei tre casi sopracitati, mappato nella pipeline di routing è rappresentato dalla rotta `/api/test`. Questo modulo agisce come un ambiente isolato per il testing dell'elaborazione dei video.

5.2.3 Middleware

All'interno del sistema, i middleware sono strutturati secondo due livelli operativi:

1. **Middleware di Trattamento Globale:** il modulo custom `detectLanguage` esegue l'auditing degli header di internazionalizzazione per impostare il contesto linguistico della risposta; il modulo `multerConfig` valida e carica nello storage gli avatar caricati dal catalogatore utilizzando la libreria "multer";
2. **Middleware di Sicurezza e Controllo Accessi (Filtri RBAC):** Implementati a protezione sussidiaria delle rotte sensibili. Il sistema si articola in una duplice barriera logica:

Filtri di Autenticazione (Authentication): Sono inclusi `auth` e `authOptional`. Questi validano la firma crittografica del JSON Web Token trasmesso nell'header `Authorization`. In caso di esito positivo, decodificano il payload iniettando l'identità dell'utente nel contesto della richiesta (`req.user`) e invocando la funzione `next()`; in caso contrario, il primo interrompe la connessione con uno stato HTTP 401 "Token non valido", mentre il secondo, essendo implementato come un middleware di autenticazione di tipo permissivo (o opzionale), non rigetta la chiamata, ma invoca programmati-

camente la funzione `next()` trattando l'utente come guest con privilegi minimi;

Filtri di Autorizzazione (Authorization): Sono inclusi `isAdmin`, `isMod`, `isCataloguer`. Questi analizzano i claim di ruolo inseriti nell'oggetto della richiesta dal filtro precedente. Se l'utente non possiede i privilegi minimi richiesti dalla rotta (privilegi di admin, moderazione o catalogazione), il middleware inibisce l'accesso restituendo uno stato HTTP 403 Forbidden, impedendo l'esecuzione di query non autorizzate sul database.

5.2.4 Utils

All'interno di questa cartella sono implementati due moduli:

Il modulo `emailService`, sfruttando la libreria `Nodemailer`, è stato configurato per gestire il flusso di recupero della password di un account registrato nella piattaforma. La configurazione dell'istanza `transporter` è interamente parametrizzata mediante variabili d'ambiente. Il sistema gestisce in modo dinamico i protocolli di cifratura del canale, discriminando a runtime l'attivazione di sessioni SSL (su porta nativa 465), TLS (su porte standard 587) o Mailtrap (porta 2525). Mediante l'invocazione del metodo asincrono `verify()`, il server esegue un test di handshake con il server SMTP remoto in fase di boot. Questo controllo previene il malfunzionamento silente delle procedure di notifica, isolando eventuali anomalie di autenticazione o di rete direttamente nei file di log del sistema. Infine il metodo `sendPasswordResetEmail` rappresenta lo strato di invio all'interno del flusso di recupero delle credenziali. Il servizio riceve un token crittografico a tempo originato dal controller di autenticazione e lo inietta all'interno di un URL strutturato per il client Angular. Il messaggio viene spedito integrando sia un corpo testuale grezzo (`text`) come fallback di accessibilità, sia un layout responsive stilizzato in HTML (`html`);

Il modulo `videoProcessor` si occupa della conversione dei flussi video nativi (formati `.mp4` o `.mkv`) in flussi di streaming adattivo conformi allo standard HLS (*HTTP Live Streaming*) con frammentazione fMP4 (*Fragmented MP4*). Sotto il profilo architetturale, l'ingegnerizzazione della pipeline si suddivide in due flussi asincroni complementari gestiti tramite il wrapper `fluent-ffmpeg` e l'esecuzione di sottoprocessi di sistema (`child_process.exec`): il metodo `processVideoHLS`, che garantisce un rapporto ottimale tra fedeltà cromatica ed efficienza del bit-rate di distribuzione; il metodo `moveMediaFile`, che segmenta in canali isolati e paralleli i flussi audio e le tracce dei sottotitoli al fine di supportare l'erogazione di contenuti in configurazione multilingua senza incorrere nell'appesantimento del canale di trasmissione.

5.2.5 Controllers

I file della cartella `/controllers`, luogo in cui risiede la logica di business vera e propria, possono essere raggruppati in base alle macro-funzionalità del sistema:

Auth, User, Propic controller: il primo gestisce la sicurezza di accesso (login), la logica di registrazione (register) e il reset o il recupero della password (`resetPassword`, `forgotPassword`); il secondo gestisce il recupero del profilo dell'utente, incluse le impostazioni sulla lingua e la lista dei preferiti; l'ultimo gestisce il recupero di tutte le foto profilo presenti nel catalogo;

Content e Streaming controller (*show, genre, season, episode, ffmpeg, discussion, comment*): il primo si occupa di gestire la ricerca degli show, di recuperare i dati della home, i dettagli di uno show e l'aggiunta o rimozione di quest'ultimo dalla lista dei preferiti di utente; il secondo di gestire il recupero di tutti i generi presenti nel DB; il terzo di recuperare i dati delle stagioni e dei relativi dettagli; il quarto si occupa di recuperare i dati degli episodi di una stagione e i relativi dettagli, di gestire l'interazione dell'utente con un episodio e la stream (generazione audio, sottotitoli, flusso del video principale); il quinto si occupa dell'elaborazione dei file multimediali caricati dal catalogatore; il sesto recupera i dati della lista di discussioni per un episodio o una specifica discussione; l'ultimo gestisce il recupero dei commenti di una discussione, la pubblicazione dei commenti, l'interazione dell'utente con i commenti, l'oscurazione e l'approvazione di un commento;

Staff & Moderation Controller (*admin, mod, cataloguer, upload*): racchiudono le operazioni privilegiate svolte dai membri dello staff, ovvero ottenere la lista degli utenti e aggiornare i ruoli (`admin`),

catalogazione (**cataloguer**) e caricamento delle immagini, file video, audio e subtrack (**upload**), moderazione delle discussioni (**mod**).

5.2.6 DB

5.2.6.1 DB.js

Questo file gestisce la connessione al database **NexuStreamSQLite** e fornisce alcune funzioni utili per eseguire query in modo asincrono usando le Promise. Svolge le seguenti funzioni: apre la connessione al database, attiva le foreign key, fornisce funzioni asincrone (**getAsync()**, **allAsync()**, **runAsync()**), inizializza il database con **initDb()** e, infine, lo azzerava con **resetDb()**.

5.2.6.2 InitDb.sql

Questo file contiene le istruzioni SQL di inizializzazione del database, ovvero la creazione di tutte le tabelle di cui sarà composto il database.

5.2.6.3 PopulateDb.js e PopulateDb.sql

Questi file contengono le istruzioni per popolare il database. Innanzitutto viene eseguito il file statico **PopulateDb.sql** che inserisce nelle tabelle i dati di Lingue, Serie, Episodi, Generi, ecc. . . Poi viene creato l'array degli utenti, viene effettuato l'hashing delle password e viene popolata la tabella degli Users. Infine vengono caricati i dati dipendenti (Commenti, interazioni, ecc. . .).

5.2.6.4 ResetDb

Questo file esegue il drop di tutte le tabelle del database.

5.2.7 Models

In questa directory si trovano tutti i models del sistema che ricevono una richiesta dai rispettivi controller ed eseguono istruzioni sql para soddisfarla.

5.2.8 Public

I contenuti multimediali binari ad alto peso specifico sono allocati all'interno di una directory dedicata denominata **/public**. Questo strato funge da Content Storage locale del server ed è esposto alla rete tramite il middleware nativo di Express **express.static()**, il quale mappa la cartella fisica del File System sotto l'endpoint virtuale **/static**.

La directory è strutturata in sotto-cartelle isolate in base al dominio d'uso e al ciclo di vita della risorsa:

/avatars: Contiene le immagini del profilo caricate dagli utenti della piattaforma.

/banners (Catalog Asset): Ospita il materiale grafico adibito alla personalizzazione visiva delle schede informative degli show;

/episode_thumbnails: Destinazione di memorizzazione delle miniature fotogrammetriche dei singoli episodi;

/show_thumbnails: Destinazione di memorizzazione delle miniature fotogrammetriche dei singoli show;

/videos: Il comparto adibito allo stoccaggio delle tracce audio/video sorgente e sottotitoli.